TP3_SdC

Integrantes del grupo

- Abad Iñaki Manuel
- Mensi Spachessi Valentín

Descripción

El trabajo práctico consistió en interactuar con la arquitectura UEFI como un entorno pre-sistema operativo. Se dividió en tres partes: la exploración del entorno virtualizado, el desarrollo y análisis estático de un binario nativo UEFI en C, y finalmente la ejecución de dicho binario directamente sobre el hardware físico (bare metal) sorteando medidas de seguridad.

Primera Parte — Exploración del entorno UEFI

Se preparó un entorno de laboratorio utilizando **QEMU** y el firmware abierto **OVMF**. A través de la UEFI Shell, se analizó cómo esta arquitectura abstrae el hardware utilizando *Handles* y *Protocolos*, lo que brinda mayor seguridad y compatibilidad frente a los puertos fijos del antiguo BIOS Legacy.

Se exploró la NVRAM interactuando con variables globales como `BootOrder` (la cual define la secuencia de arranque) y se inyectó exitosamente una variable de prueba (`TestSeguridad`). Además, se realizó el footprinting del sistema volcando el mapa de memoria y la lista de dispositivos PCI. En el mapa de memoria se identificaron regiones `RuntimeServicesCode`, las cuales no se borran al cargar el sistema operativo y son objetivos clave para inyectar bootkits persistentes.

Segunda Parte — Desarrollo y Análisis en C

En esta fase se desarrolló una aplicación en C nativa para UEFI. Dado que se trata de un entorno *freestanding* sin librerías estándar (como libc), se debió utilizar el protocolo `ConOut` (`SystemTable->ConOut->OutputString`) del firmware para poder imprimir texto en pantalla en lugar de un `printf` convencional. El código fue compilado al formato PE/COFF de Windows (`aplicacion.efi`).

El binario incluyó una instrucción de breakpoint estático (`0xCC`). Al analizar el archivo resultante con el descompilador **Ghidra**, se observó cómo este traduce las rutinas UEFI a llamadas genéricas por falta de firmas, e interpreta por defecto el valor `0xCC` como el entero con signo `-52` (opcode crítico en ciberseguridad utilizado para anti-debugging y hooks).

Finalmente, el binario fue cargado y ejecutado exitosamente dentro del emulador QEMU. A continuación, algunas capturas del proceso de exploración:

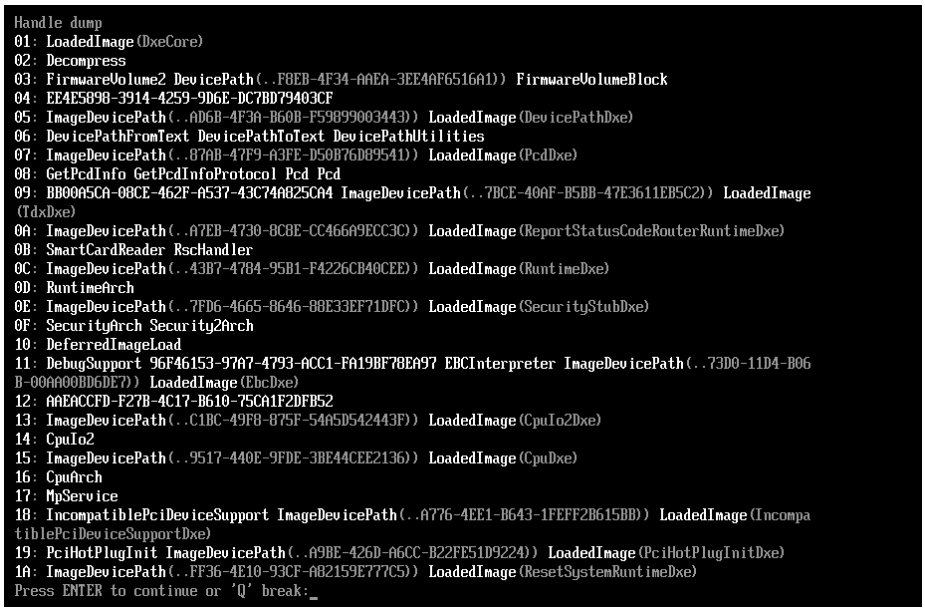


Figure 1: Listado de identificadores y protocolos en lugar de direcciones físicas fijas

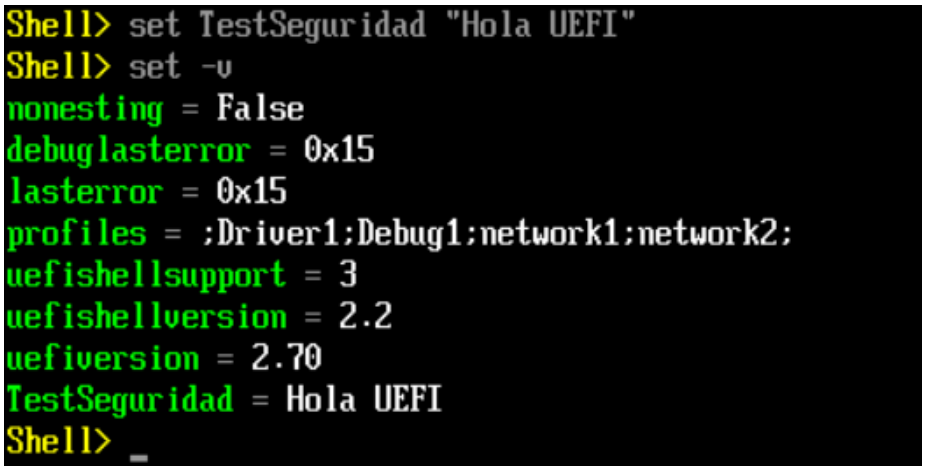


Figure 2: Inyección de una variable en el entorno no volátil

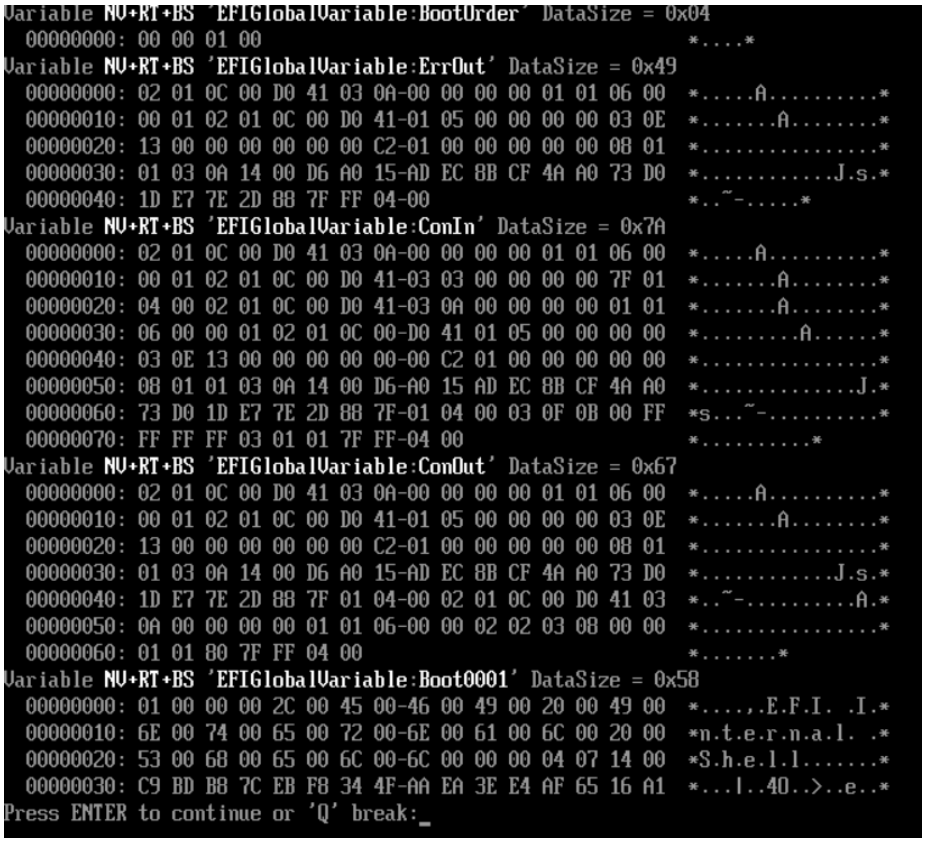


Figure 3: Variables predefinidas por el firmware

```
ACPI_Recl  000000001F76D000-000000001F77EFFF 0000000000000012 000000000000000F
ACPI_NVS   000000001F77F000-000000001F7FEFFF 0000000000000080 000000000000000F
BS_Data    000000001F7FF000-000000001FDFFFFF 0000000000000601 000000000000000F
Available  000000001FE00000-000000001FE76FFF 0000000000000077 000000000000000F
BS_Data    000000001FE77000-000000001FE96FFF 0000000000000020 000000000000000F
BS_Code    000000001FE97000-000000001FEC9FFF 0000000000000033 000000000000000F
Press ENTER to continue or 'Q' break:
BS_Data    000000001FECA000-000000001FEDAFFF 0000000000000011 000000000000000F
BS_Code    000000001FEDB000-000000001FEF3FFF 0000000000000019 000000000000000F
RT_Data    000000001FEF4000-000000001FF77FFF 0000000000000084 800000000000000F
ACPI_NVS   000000001FF78000-000000001FFFFFFF 0000000000000088 000000000000000F
Reserved   000000FD00000000-000000FFFFFFFFFF 0000000000300000 0000000000000000

  Reserved  :      3,145,856 Pages (12,885,426,176 Bytes)
  LoaderCode:            217 Pages (888,832 Bytes)
  LoaderData:              0 Pages (0 Bytes)
  BS_Code   :            850 Pages (3,481,600 Bytes)
  BS_Data   :          8,470 Pages (34,693,120 Bytes)
  RT_Code   :            256 Pages (1,048,576 Bytes)
  RT_Data   :            581 Pages (2,379,776 Bytes)
  ACPI_Recl :             18 Pages (73,728 Bytes)
  ACPI_NVS  :            513 Pages (2,101,248 Bytes)
  MMIO      :              0 Pages (0 Bytes)
  MMIO_Port :              0 Pages (0 Bytes)
  PalCode   :              0 Pages (0 Bytes)
  Unaccepted:              0 Pages (0 Bytes)
  Available :        119,943 Pages (491,286,528 Bytes)
  Persistent:              0 Pages (0 Bytes)
             --------------
Total Memory:            511 MB (535,953,408 Bytes)
Shell> _
```

Figure 4: Zonas de memoria, incluyendo los Runtime Services

```
Shell> pci -b
   Seg  Bus  Dev  Func
   ---  ---  ---  ----
    00   00   00    00 ==> Bridge Device - Host/PCI bridge
                Vendor 8086 Device 1237 Prog Interface 0
    00   00   01    00 ==> Bridge Device - PCI/ISA bridge
                Vendor 8086 Device 7000 Prog Interface 0
    00   00   01    01 ==> Mass Storage Controller - IDE controller
                Vendor 8086 Device 7010 Prog Interface 80
    00   00   01    03 ==> Bridge Device - Other bridge type
                Vendor 8086 Device 7113 Prog Interface 0
    00   00   02    00 ==> Display Controller - VGA/8514 controller
                Vendor 1234 Device 1111 Prog Interface 0
Shell> _
```

Figure 5: Listado de hardware PCI detectado

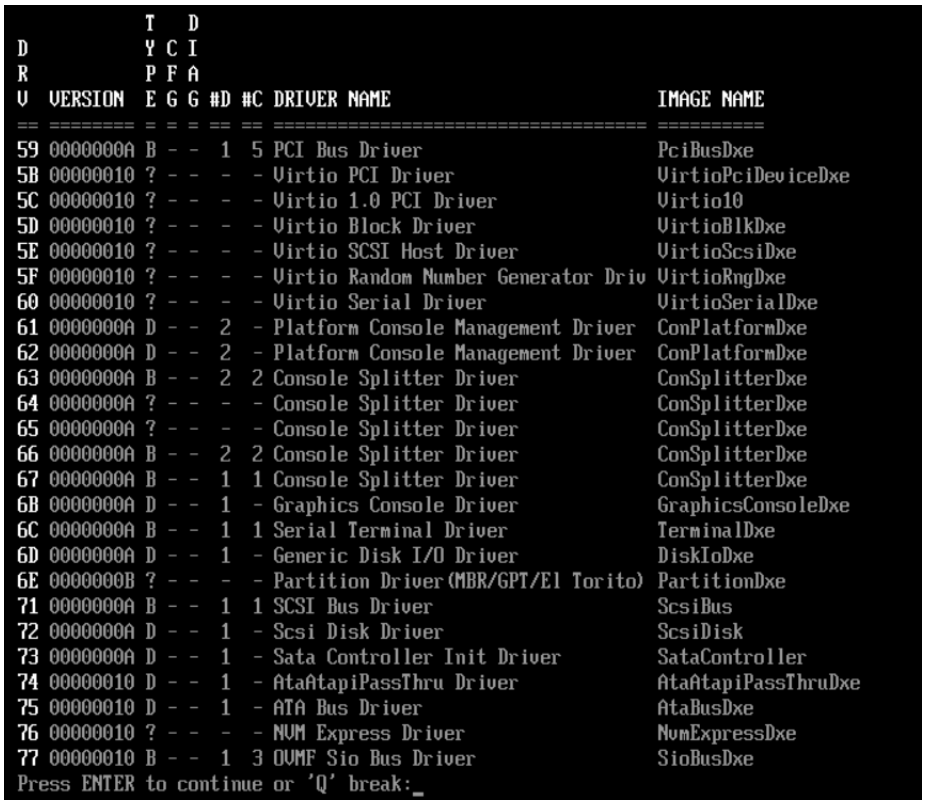
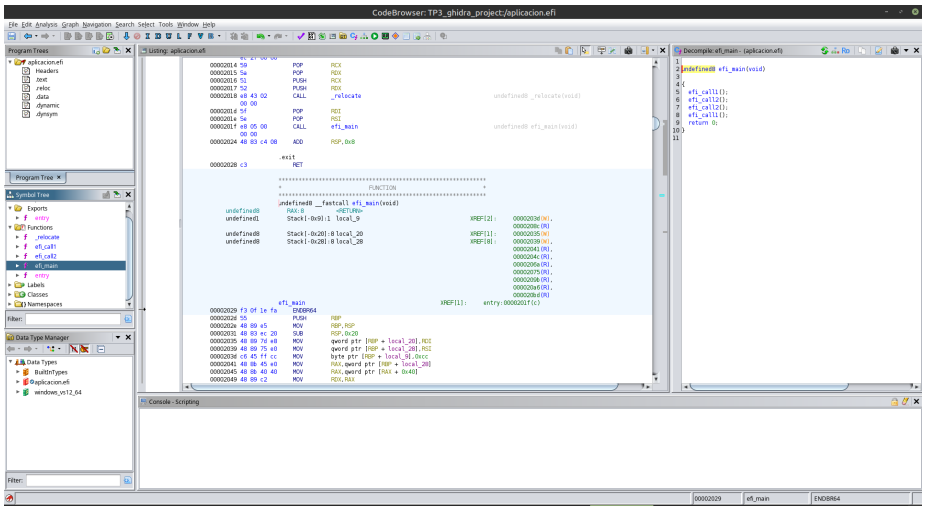
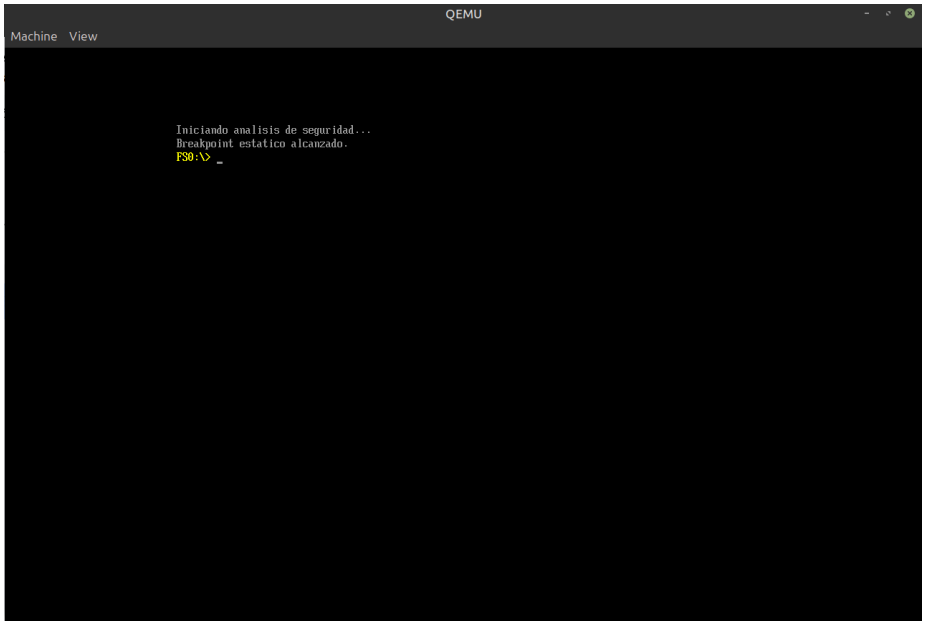


Figure 6: Drivers cargados en el entorno pre-OS



Traducción del ensamblador al pseudocódigo en Ghidra, visualizando la inicialización del byte `0xcc`.



El breakpoint es evaluado correctamente de forma dinámica.

Tercera Parte — Ejecución en Hardware Real

Para la última iteración, se preparó un pendrive en formato `FAT32` con la estructura estandarizada `/EFI/BOOT/BOOTX64.EFI` (conteniendo la UEFI Shell oficial) y el binario desarrollado.

Se configuró la BIOS de una notebook Haier, de tal forma que se **desactivó el Secure Boot**. Este paso fue obligatorio, ya que los binarios generados no poseían una firma criptográfica autorizada y hubieran sido rechazados por el sistema de validación del hardware.

La aplicación se ejecutó exitosamente de forma directa sobre la placa base, sin ningún sistema operativo subyacente.

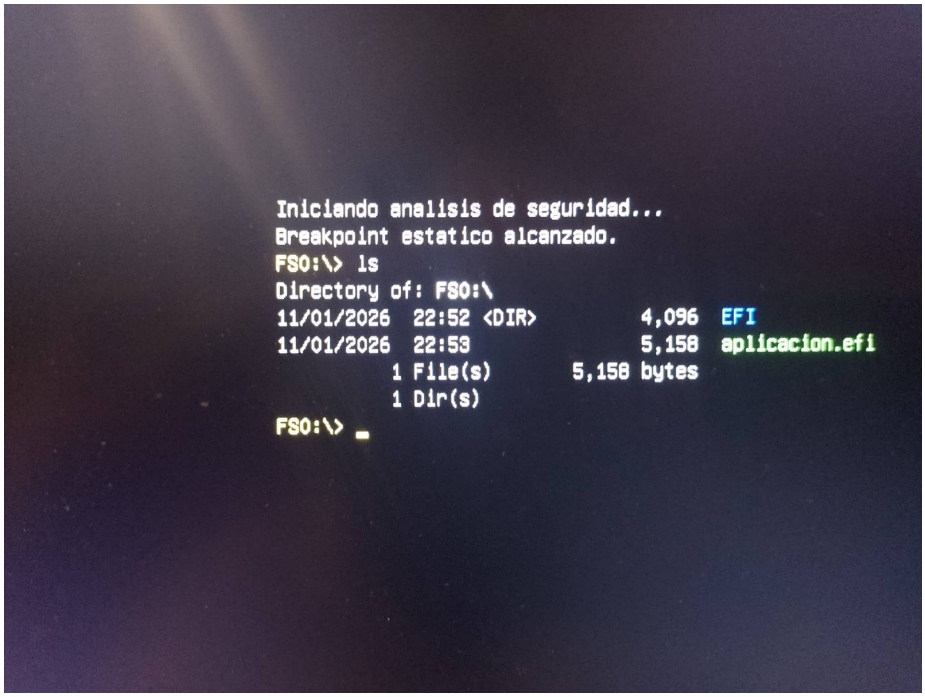


Figure 7: Ejecución en Hardware Físico

Salida de la aplicación en la pantalla de la notebook

Tecnologías utilizadas

- C
- Assembly x86-64
- GNU-EFI / TianoCore
- QEMU / OVMF
- Ghidra
- GCC